Chapter 1. Prediction Problems and Evaluation Design

Predictive Modeling, Lecture 1 (Is the score trustworthy? Splits, CV, leakage, OLS, logistic regression)

Contents

1 Prediction as a Rule for New Cases **2**

1.1 The prediction problem . . . 2

1.2 The prediction time decides which variables are usable . . . 2

1.3 Regression and classification . . . 2

1.4 Prediction versus causal inference . . . 3

2 Loss, Risk and Generalization **3**

2.1 Loss translates decision costs into numbers . . . 3

2.2 Training score versus deployment risk . . . 4

3 Splitting the Data **4**

3.1 Three roles . . . 4

3.2 The split must preserve the data-generating structure . . . 4

3.3 Reproducibility . . . 5

4 Information Leakage **5**

5 K-fold Cross-Validation **5**

6 Metrics are the Language of Decisions **6**

6.1 Regression metrics . . . 6

7 Classification: Probabilities, Thresholds, Metrics **7**

7.1 Probability first, action second . . . 7

7.2 Threshold and confusion matrix . . . 7

7.3 Three questions about one probability . . . 7

7.4 Choosing the threshold from costs . . . 7

8 The Linear Baseline: OLS **8**

8.1 Least squares . . . 8

8.2 Predictive versus causal reading of coefficients . . . 8

8.3 Residual diagnostics point to the next model . . . 8

9 Logistic Regression **8**

9.1 From linear score to probability . . . 8

9.2 Log loss . . . 9

10 Comparing Candidates Fairly **9**

Roadmap of the predictive-modeling part

Lecture	Key question	Models and concepts
1	Can the score be trusted?	splits, CV, leakage, OLS, logistic regression
2	Many correlated variables?	Ridge, Lasso, Elastic Net
3	How to learn nonlinear relations?	decision trees, bagging, random forest
4	Strong models for tabular data?	GBM, XGBoost, LightGBM, CatBoost
5	When are neural nets useful?	MLP, minibatch, Adam, regularization, TabPFN/TabICL
6	How to combine many models?	AutoGluon, OOF, bagging, stacking

Models may change; the prediction time, split, pipeline and final-test rule must not.

1 Prediction as a Rule for New Cases

1.1 The prediction problem

Definition: prediction rule

From observed data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ with $x_i \in \mathbb{R}^p$ we build a rule

$$\hat{f} : \mathcal{X} \to \mathcal{Y}, \qquad \hat{y}_{\text{new}} = \hat{f}(x_{\text{new}}).$$

x_i is the feature vector already observed at the **prediction time**, y_i the target observed later. The object of interest is the **prediction error on new cases**, not the fit to $\mathcal{D}$.

1.2 The prediction time decides which variables are usable

To predict next month's churn at the end of this month, nothing after the end of the month may be a feature.

Role of a column	Question at prediction time	Churn example
Feature X_j	Is it already known at that moment?	recent usage, contract length, # complaints
Target Y	What do we predict?	churn next month
Identifier (ID)	Does it carry anything beyond row identity?	customer ID, account serial number
Post-event variable	Does it arise only after the outcome?	reason for cancellation, refund after cancellation

Post-event variables raise the score but do not exist at prediction time: never use them.

1.3 Regression and classification

Task	Target	Direct model output	Focus of evaluation
Regression	continuous Y	$\hat{y}(x)$, conditional mean	average error and large errors
Binary classification	$Y \in \{0, 1\}$	$\hat{p}(x) = P(Y = 1 \mid X = x)$	ranking, probability, threshold errors
Multiclass	$Y \in \{1, \dots, C\}$	$(\hat{p}_1, \dots, \hat{p}_C)$	per-class errors and probability quality

A classifier outputs a **probability, not a label**; the action is a separate decision (§7).

1.4 Prediction versus causal inference

Prediction: $\hat{y} = \hat{f}(x)$	Causal inference: $\mathbb{E}\{Y(1) - Y(0)\}$
Who is likely to churn? Small loss on new cases is the goal.	How much did a coupon reduce churn? Identification assumptions and bias control are central.
Correlated (non-causal) variables can be useful features.	Confounders and post-treatment variables need care.

A good predictive coefficient never justifies a causal claim.

2 Loss, Risk and Generalization

Motivation. Risk is the expected loss on unseen data, which training loss underestimates.

2.1 Loss translates decision costs into numbers

Definition: loss and risk

A loss $L(y, f(x)) \geq 0$ measures the cost of predicting $f(x)$ when the truth is y. The **risk** in the deployment environment is

$$R(f) = \mathbb{E}_{(X,Y)\sim P_{\text{deploy}}}[L\{Y, f(X)\}].$$

$R(f)$ cannot be observed directly. The **training risk** is $\widehat{R}_{\text{train}}(f) = \frac{1}{n}\sum_{i=1}^{n} L\{y_i, f(x_i)\}$, and the **generalization gap** is $\text{Gap}(f) = R(f) - \widehat{R}_{\text{train}}(f)$.

Fix the loss *before* the model (overstock vs. stock-out, missed defaulter vs. rejected good customer); accuracy is only one loss.

Theorem: what each loss targets

Let $f^\star = \arg\min_f R(f)$ over all measurable f. Pointwise in x:

1. squared loss $(y - f)^2$: $f^\star(x) = \mathbb{E}[Y \mid X = x]$ (conditional mean);
2. absolute loss $|y - f|$: $f^\star(x) = \text{median}(Y \mid X = x)$;
3. 0–1 loss for $Y \in \{0, 1\}$: $f^\star(x) = \mathbb{I}\{P(Y = 1 \mid x) \geq 1/2\}$ (Bayes classifier);
4. log loss $-[y \log q + (1 - y)\log(1 - q)]$ over $q \in (0, 1)$: $q^\star(x) = P(Y = 1 \mid X = x)$ (a *proper* scoring rule).

Proof [Supplement]. $R(f) = \mathbb{E}_X[\mathbb{E}\{L(Y, f(X)) \mid X\}]$, so minimize $g(c) = \mathbb{E}[L(Y, c) \mid X = x]$ for each x.

(1) $g(c) = \text{Var}(Y \mid x) + (\mathbb{E}[Y \mid x] - c)^2$.

(2) $g'(c) = 2P(Y \leq c \mid x) - 1$ (right derivative): < 0 below the conditional median, ≥ 0 from it on.

(3) $g(1) = 1 - \pi(x)$, $g(0) = \pi(x)$ with $\pi(x) = P(Y = 1 \mid x)$; choose 1 iff $\pi \geq \frac{1}{2}$.

(4) $g(q) = -\pi \log q - (1 - \pi)\log(1 - q)$, $g'(q) = -\pi/q + (1 - \pi)/(1 - q) = 0 \iff q = \pi$, and $g'' > 0$. ■

2.2 Training score versus deployment risk

Fitting minimizes $\widehat{R}_{\text{train}}$, which complex models drive near zero; $R(f)$ needs an **independent validation procedure**. The gap becomes unstable with large model searches, splits ignoring time/groups, full-data preprocessing, and distribution shift.

Theorem: bias–variance decomposition

Let $Y = f(x) + \varepsilon$ with $\mathbb{E}\varepsilon = 0$, $\text{Var}\varepsilon = \sigma^2$, and let $\hat{f}$ be fitted on a random training set independent of the new (x, Y). Then

$$\mathbb{E}[(Y - \hat{f}(x))^2] = \underbrace{\left(f(x) - \mathbb{E}\hat{f}(x)\right)^2}_{\text{Bias}^2} + \underbrace{\text{Var}\hat{f}(x)}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{irreducible}}.$$

Proof [Supplement]. $Y - \hat{f} = \varepsilon + (f - \mathbb{E}\hat{f}) + (\mathbb{E}\hat{f} - \hat{f})$; the terms are pairwise uncorrelated ($\varepsilon \perp \hat{f}$ with mean 0; $f - \mathbb{E}\hat{f}$ constant; $\mathbb{E}(\mathbb{E}\hat{f} - \hat{f}) = 0$). Square and take expectations. ■

Training error falls monotonically with complexity; test error is U-shaped (**underfitting**: high bias; **overfitting**: high variance).

3 Splitting the Data

3.1 Three roles

Part	Used for	Never used for
Training part	fitting model parameters	—
Internal validation (holdout or CV)	choosing candidates and tuning	—
Test set	reporting final generalization once	choosing models

3.2 The split must preserve the data-generating structure

Data structure	Recommended split	Danger of a random split
independent rows, class imbalance	stratified split	minority-class proportion may differ by chance
clustered (customers, hospitals, households)	group split	the same group appears in training and test
time order	forward (time-ordered) split	future information leaks into training
duplicates / near-duplicates	deduplicate, then split	nearly identical rows on both sides

The validation split follows from the *unit of a new case*.

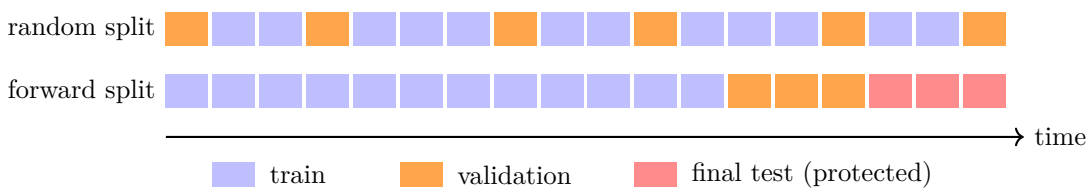


Figure 1: A random split trains on the future of its validation cases; a forward split trains on the past and protects the latest period as the final test.

3.3 Reproducibility

Fix: split rule and random seed; feature list and target definition; handling of missing values, categories, scaling; code computing metrics and thresholds.

Record: extraction time, data version, number of rows; class ratios and number of groups in each set; candidate models and hyperparameter grids; fold-by-fold variability, not only the CV mean.

4 Information Leakage

Definition: leakage

Leakage is any channel through which information unavailable at prediction time, or information from the validation/test part, enters the fitting procedure.

Type	Example	Why it is a problem
Target leakage	cancellation reason or default-handling result used as a feature	does not exist at prediction time
Preprocessing leakage	mean imputation / standardization on the full data	peeks at the validation/test distribution
Temporal leakage	future averages, after-the-fact aggregates	cannot reproduce a past decision
Duplicate / group leakage	same customer, document or product on both sides	the "new" case was effectively already seen

Example: why preprocessing must be learned inside each fold

Screening on the full data ("keep the 20 features most correlated with y") lets a model on pure noise score high in CV, since screening has seen the validation targets. Fix: re-fit every data-dependent step (imputation, scaling, screening, encoding) inside each training fold.

5 K-fold Cross-Validation

Definition: K-fold CV

Split the development data into folds $V_1, \ldots, V_K$. For each k fit $\hat{f}^{(-k)}$ on the data without V_k

and compute

$$\widehat{R}_k(f) = \frac{1}{|V_k|} \sum_{i \in V_k} L\{y_i, \hat{f}^{(-k)}(x_i)\}, \qquad \widehat{R}_{\mathrm{CV}}(f) = \frac{1}{K} \sum_{k=1}^{K} \widehat{R}_k(f).$$

Each observation is validated once and trained on $K-1$ times.

Remark: fold variability and its standard error

Report $\widehat{R}_{\mathrm{CV}}$ with $\mathrm{SE} = \mathrm{sd}(\widehat{R}_1, \ldots, \widehat{R}_K)/\sqrt{K}$ (rough, usually too small; used by the 1-SE rule of Ch. 3).

Algorithm: selection with CV, reporting with the test set

1. Split off the test set first and lock it away.
2. On the remaining development data, define the pipeline (preprocessing + candidate model).
3. Choose candidates and hyperparameters using CV scores and fold variability.
4. Refit the chosen pipeline on the whole development data.
5. Report final performance, error patterns and operating cost on the test set.

6 Metrics are the Language of Decisions

A metric should track operational loss and be fixed across candidates. Ask first *which errors are more expensive.*

6.1 Regression metrics

Metric	Definition	Interpretation
MAE	$\frac{1}{n}\sum_i \lvert y_i - \hat{y}_i \rvert$	all errors linearly; linked to median prediction
MSE	$\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2$	squared penalty on large errors; linked to mean prediction
RMSE	$\sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}$	same unit as y; emphasizes large errors
R^2	$1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$	improvement in squared error over the mean baseline

Theorem: $\mathrm{MAE} \le \mathrm{RMSE} \le \sqrt{n}\,\mathrm{MAE}$

For residuals $e_i = y_i - \hat{y}_i$, $\frac{1}{n}\sum |e_i| \le \left(\frac{1}{n}\sum e_i^2\right)^{1/2}$, with equality iff all $|e_i|$ are equal.

Proof [Supplement]. Cauchy–Schwarz: $\left(\sum |e_i| \cdot 1\right)^2 \le n \sum e_i^2$. The upper bound follows from $\sum e_i^2 \le \left(\sum |e_i|\right)^2$. ∎

RMSE − MAE grows with the dispersion of $|e_i|$. Prefer RMSE when large errors are costly, MAE to limit outlying targets.

7 Classification: Probabilities, Thresholds, Metrics

7.1 Probability first, action second

A classifier outputs $\hat{p}(x) = P(Y = 1 \mid X = x)$; one $\hat{p} = 0.18$ can mean different actions under different costs. Calibration asks whether cases predicted at 0.18 churn at about 18%.

7.2 Threshold and confusion matrix

$$\hat{y}_t(x) = \mathbb{I}\{\hat{p}(x) \geq t\}, \qquad \begin{array}{l|cc} & \text{actual } + & \text{actual } - \\ \hline \text{predicted } + & TP & FP \\ \text{predicted } - & FN & TN \end{array}$$

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}, \qquad \text{accuracy} = \frac{TP + TN}{n}.$$

Lowering t raises recall and false positives.

Example: accuracy under imbalance

50 churners among 1,000: predicting "no churn" for all gives accuracy 95%, recall 0. Use precision, recall and costs.

7.3 Three questions about one probability

Ranking	Reliability of the probability	Action
Whom to look at first? ROC-AUC, PR-AUC	Does 0.30 mean about 30%? Brier score, log loss, calibration plot	Where to place the threshold? precision/recall at t, cost

Theorem: ROC-AUC is a ranking probability

For a score s, $\text{AUC} = P(s(X^+) > s(X^-)) + \frac{1}{2}P(s(X^+) = s(X^-))$, where X^+, X^- are independent draws from the positive and negative classes. Empirically AUC equals the proportion of (positive, negative) pairs ranked correctly (Mann–Whitney statistic).

Proof [Supplement]. ROC plots $\text{TPR}(t) = P(s^+ > t)$ against $\text{FPR}(t) = P(s^- > t)$, so $\int \text{TPR}\, d\text{FPR} = \int P(s^+ > t)\, dF^-(t) = P(s^+ > s^-)$ for continuous scores (ties contribute half). ■

PR-AUC suits rare positives. Brier $\frac{1}{n}\sum(y_i - \hat{p}_i)^2$; log loss punishes confident errors. AUC is invariant to monotone transforms, so it ignores calibration.

7.4 Choosing the threshold from costs

Theorem: cost-optimal threshold

Let a false negative cost C_{FN} and a false positive cost C_{FP} (correct decisions cost 0). If $\hat{p}(x)$ is calibrated, acting positive minimizes expected cost iff

$$\hat{p}(x)\, C_{FN} > \{1 - \hat{p}(x)\}\, C_{FP} \iff \hat{p}(x) > t^\star = \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

Proof [Supplement]. Positive: cost C_{FP} w.p. $1-\hat{p}$; negative: cost C_{FN} w.p. $\hat{p}$. Compare and rearrange. ■

Larger C_{FN} lowers $t^\star$; $t^\star = 1/2$ only for equal costs.

8 The Linear Baseline: OLS

8.1 Least squares

$$\hat{y}_i = \beta_0 + x_i^\top \beta, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{n} (y_i - \beta_0 - x_i^\top \beta)^2.$$

With the design matrix X including the intercept column,

Derivation: normal equations

$Q(\beta) = \|y - X\beta\|^2 = y^\top y - 2\beta^\top X^\top y + \beta^\top X^\top X\beta$, so $\nabla Q = -2X^\top y + 2X^\top X\beta = 0$ gives $X^\top X\hat{\beta} = X^\top y$, $\hat{\beta} = (X^\top X)^{-1}X^\top y$ if invertible; Hessian $2X^\top X \succeq 0$ gives a global minimum. $\hat{y} = Hy$, $H = X(X^\top X)^{-1}X^\top$, is the orthogonal projection onto the column space: $X^\top(y - \hat{y}) = 0$.

Nearly singular $X^\top X$ means unstable coefficients; see regularization (Ch. 3).

8.2 Predictive versus causal reading of coefficients

Predictively, β_j is the change in prediction per unit of X_j with others fixed; under multicollinearity it wobbles. A causal reading also needs confounder control, no post-treatment variables, and no selection or measurement bias.

8.3 Residual diagnostics point to the next model

Observed phenomenon	Meaning	Next candidates
curved residual pattern	missed nonlinearity / interaction	transformations, splines, trees (Ch. 4)
variance changing with $\hat{y}$	heteroscedasticity, scale issue	transformations, robust loss, segment-wise evaluation
a few huge residuals	outliers, label errors	Huber loss, data checks (Ch. 2)
unstable coefficients with correlated features	multicollinearity, high dimension	Ridge, Lasso, Elastic Net (Ch. 3)

9 Logistic Regression

9.1 From linear score to probability

$$\eta(x) = \beta_0 + x^\top \beta, \qquad \hat{p}(x) = \sigma\{\eta(x)\} = \frac{1}{1+\exp\{-\eta(x)\}}, \qquad \log\frac{p(x)}{1-p(x)} = \beta_0 + x^\top\beta.$$

β_j is the change of the **log-odds** per unit of X_j, $\exp(\beta_j)$ the **odds ratio**, and $\partial p/\partial x_j = \beta_j p(1-p)$.

9.2 Log loss

$$\ell(y,p) = -\big[y\log p + (1-y)\log(1-p)\big], \qquad \hat\beta = \arg\min_{\beta} \sum_{i=1}^{n} \ell\{y_i, \sigma(\beta_0 + x_i^\top \beta)\}.$$

Confident wrong probabilities ($p \approx 0$ for a positive) get huge penalties.

Theorem: gradient and convexity of the logistic log loss

With $p_i = \sigma(x_i^\top \beta)$ (intercept absorbed), $J(\beta) = \sum_i \ell(y_i, p_i)$ satisfies

$$\nabla J(\beta) = X^\top (p - y), \qquad \nabla^2 J(\beta) = X^\top W X, \quad W = \mathrm{diag}\{p_i(1-p_i)\} \succeq 0,$$

so J is convex and minimizing log loss equals maximum likelihood for the Bernoulli model. Newton's method is iteratively reweighted least squares.

Proof [Supplement]. $\sigma' = \sigma(1-\sigma)$, so $\partial\ell/\partial\eta = -y(1-p) + (1-y)p = p - y$; hence $\nabla J = \sum_i (p_i - y_i)x_i$ and $\nabla^2 J = \sum_i p_i(1-p_i)x_i x_i^\top$. The Bernoulli log-likelihood is $\sum_i \{y_i \log p_i + (1-y_i)\log(1-p_i)\} = -J$. ■

Remark: separation

Under perfect separation J decreases as $\|\beta\| \to \infty$ and no MLE exists; penalization (Ch. 3) fixes this.

10 Comparing Candidates Fairly

Criterion	Conditions to control
same split	all candidates use the same training/CV/test structure
same feature definition	same prediction time, missing-value and category handling
same metric	primary and secondary metrics for selection fixed in advance
same compute budget	record search range, time and number of iterations
same reporting	show CV mean, variability, test score, complexity and operating constraints together

Summary

- Define the target and the prediction time first.
- The goal is low generalization risk in deployment, not low training error.
- The split must preserve the data-generating structure: time, groups, duplicates.
- CV is for selection, the test set for the final report; preprocessing is learned inside each training fold.
- Metrics, thresholds and baselines are chosen from decision costs before algorithms.